

20
Total Marks

30 min
Time

MCQ
Type

3 — OOP
Day

Instructions

Har question ke liye ek sahi answer choose karo. Code questions mein mentally trace karo.

Pehle khud solve karo — phir answer key check karo (last page pe hai).

★ marked questions interview mein often pooche jaate hain — extra dhyan do.

Q1. [Prototype Chain] ★ JavaScript mein prototype chain kab end hoti hai?

- (A) jab koi object na mile
- (B) Object.prototype ke baad — __proto__ null hota hai
- (C) Function.prototype pe
- (D) jab hasOwnProperty false return kare

Q2. [Prototype] Object.create(null) se banaya object kaise alag hota hai?

- (A) Bilkul same hota hai
- (B) Koi prototype nahi hota — Object.prototype methods bhi nahi
- (C) Immutable hota hai
- (D) Only string keys allow karta hai

Q3. [Prototype] ★ for...in aur Object.keys() mein farq kya hai?

- (A) Koi farq nahi
- (B) Object.keys() sirf own enumerable properties deta hai; for...in inherited bhi
- (C) for...in faster hai
- (D) Object.keys() inherited properties bhi include karta hai

Q4. [Prototype] d.hasOwnProperty('speak') false return kare to matlab?

- (A) 'speak' exist nahi karta
- (B) 'speak' prototype chain mein hai, instance pe nahi
- (C) d undefined hai
- (D) Property enumerable nahi hai

Q5. [ES6 Class] ★ class keyword ke baare mein kaunsa statement SAHI hai?

- (A) class function se bilkul alag mechanism hai
- (B) class sirf syntactic sugar hai — under the hood prototype use hota hai
- (C) class mein hoisting hoti hai
- (D) class methods instances mein copy hote hain

Q6. [ES6 Class] constructor() method kab automatically call hota hai?

- (A) Jab class import ho
- (B) Jab new keyword se object banaye
- (C) Jab method call ho
- (D) Manually call karna padta hai

Q7. [Class Fields] ★ class C { #x = 0; } — #x kya hai?

- (A) Comment
- (B) Private class field — sirf class ke andar accessible
- (C) Static property
- (D) Protected property

Q8. [ES6 Class] Method chaining kaise possible hota hai?

- (A) Arrow functions use karo
- (B) Method ke end mein return this; likho
- (C) Static methods se
- (D) Getters use karo

Q9. [Inheritance] ★ Child class constructor mein this se pehle super() nahi call kiya to?

- (A) Warning print hogi
- (B) ReferenceError: Must call super before accessing 'this'
- (C) undefined milega
- (D) Parent constructor ignore hoga

Q10. [Inheritance] super.method() kab use karte hain?

- (A) Parent class ka constructor call karne ke liye
- (B) Method mein parent class ke overridden method ko call karne ke liye
- (C) Static methods call karne ke liye
- (D) Private fields access karne ke liye

Q11. [Inheritance] ★ Ye code kya print karega? class A { greet(){ return 'A'; }} class B extends A { greet(){ return super.greet() + 'B'; }} new B().greet();

- (A) 'A'
- (B) 'B'
- (C) 'AB'
- (D) Error

Q12. [Inheritance] extends ke baad child class ka prototype chain kya hoga?

- (A) Child.prototype → Object.prototype
- (B) Child.prototype → Parent.prototype → Object.prototype
- (C) Child → Parent
- (D) Child.prototype → null

Q13. [Static] Static method ko kaise call karte hain?

- (A) instance.method()
- (B) ClassName.method()
- (C) this.method()
- (D) new ClassName.method()

Q14. [Getters/Setters] ★ Getter aur regular method mein kya fark hai?

- (A) Koi farq nahi
- (B) Getter property ki tarah access hota hai — () nahi lagta
- (C) Getter faster hai
- (D) Getter sirf private fields pe kaam karta hai

Q15. [Getters/Setters] Setter mein validation karne ka kya faida hai?

- (A) Code slow hota hai
- (B) Invalid data directly object mein store nahi hota — data integrity
- (C) Memory save hota hai
- (D) Sirf private fields ke liye zarori hai

Q16. [Mixins] Mixin pattern kyun use karte hain?

- (A) Deep inheritance avoid karne ke liye — multiple behaviors compose karna
- (B) Performance improve karne ke liye
- (C) Private fields banana ke liye
- (D) Static methods add karne ke liye

Q17. [Mixins] const M = (Base) => class extends Base {}; — ye kya hai?

- (A) Regular class declaration
- (B) Mixin function — Base class mein behavior add karta hai

- (C) Abstract class
- (D) Factory function

Q18. [Prototype] ★ Ye kya return karega? `const obj = { x:1 }; const child = Object.create(obj); console.log(child.x); console.log(child.hasOwnProperty('x'));`

- (A) 1, true
- (B) 1, false — x inherited hai, own nahi
- (C) undefined, false
- (D) Error

Q19. [ES6 Class] ★ class vs constructor function — kaunsa statement GALAT hai?

- (A) Dono prototype chain use karte hain
- (B) class methods prototype pe hote hain, instances mein copy nahi
- (C) class hoisted hoti hai jaise function
- (D) class strict mode enforce karta hai

Q20. [Prototype] instanceof operator kya check karta hai?

- (A) Object ka type
- (B) Object ke prototype chain mein given constructor ka prototype hai ya nahi
- (C) Object ka class name
- (D) Property exist karta hai ya nahi

Answer Key

Q1: (B)	Q2: (B)	Q3: (B)	Q4: (B)	Q5: (B)
Q6: (B)	Q7: (B)	Q8: (B)	Q9: (B)	Q10: (B)
Q11: (C)	Q12: (B)	Q13: (B)	Q14: (B)	Q15: (B)
Q16: (A)	Q17: (B)	Q18: (B)	Q19: (C)	Q20: (B)

Scoring Guide

20/20 (100%) — Outstanding! OOP concepts crystal clear hain.

15-19 (75-95%) — Good! ★ marked questions ke answers review karo.

10-14 (50-70%) — Prototype chain aur inheritance ke theory notes dobara padhein.

Below 10 (<50%)— Day 3 topics ek baar aur karo, mentor se discuss karo.